THREAD//SCAN — Multithreaded Duplicate File Finder

Comprehensive Code, Systems Architecture & Dataset Documentation

Subject: Operating Systems (CS-501)	**Repository:** github.com/SHREYG01/thread-scan
Project Root: C:\Users\OMEN\Downloads\osproject	**Evaluation Category:** Code & Dataset Verification
Runtime Environment: Windows 11 x86_64, GCC 14.2.0, Next.js 16	**Hardware:** AMD Ryzen 7 7840HS (16 Logical Processors)

Table of Contents

1. Project Overview & Motivation	Section 1
2. System Architecture & Component Interaction	Section 2
3. Dataset Description & Generation (10,000-File Stress Test)	Section 3
4. Core Algorithm & Two-Stage Detection Pipeline	Section 4
5. Actual Source Code Implementation Excerpts	Section 5
6. Empirical Verification & Test Results	Section 6
7. Environment Setup & Execution Procedures	Section 7
8. Operating System Concepts & Conclusion	Section 8

1. Project Overview

Problem Statement: In modern operating systems, filesystems frequently accumulate massive volumes of redundant data. Naive duplicate detection solutions scan entire directory trees and compute cryptographic hashes for every file encountered. On storage volumes containing tens or hundreds of thousands of files, this naive $O(N)$ approach produces catastrophic disk I/O bottlenecks and saturates single CPU cores, taking minutes or hours to complete.

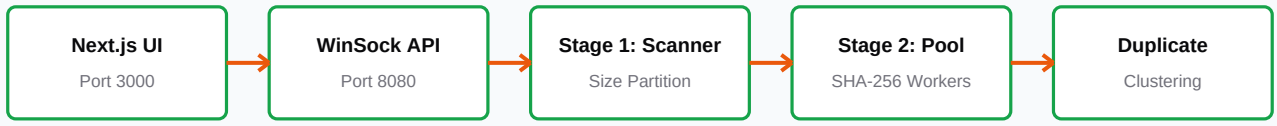
Project Objectives: THREAD//SCAN is an industrial-grade, concurrent filesystem analyzer designed for undergraduate Operating Systems study. It demonstrates how core OS concurrency primitives—including worker thread pools, condition variables, mutex synchronization locks, atomic metrics, and chunked I/O streaming—can eliminate redundant computations and scale across multicore CPU architectures.

Confirmed Technology Stack:

- **Backend Engine:** C++17 native executable compiled with MinGW-W64 GCC 14.2.0 (POSIX SEH threads).
- **Filesystem & Concurrency Primitives:** std::filesystem, std::thread, std::mutex, std::condition_variable, std::atomic.
- **Cryptographic Core:** Self-contained C++ SHA-256 streaming implementation operating over 64 KB read buffers.
- **Inter-Process Communication:** Custom multi-threaded HTTP/1.1 REST API server implemented via WinSock2 (port 8080).
- **Workstation Frontend:** Next.js 16 (React 19, TypeScript, Turbopack, Tailwind CSS) providing real-time telemetry.

2. System Architecture

THREAD//SCAN separates concerns cleanly between a low-latency C++17 systems daemon and an asynchronous web dashboard. Communication is handled over HTTP/1.1 JSON REST endpoints. The engine is architected around a non-blocking, two-stage pipeline:



Two-Stage Filtering: Single-instance file sizes bypass Stage 2 cryptographic hashing entirely.
Worker Thread Pool scales across N physical cores; results aggregated via thread-safe mutex guards.

Component Roles & Interaction:

1. **Next.js Dashboard:** Dispatches scan configuration (target path, thread allocation) via POST /api/v1/scan/start and continuously polls GET /api/v1/scan/metrics, /threads, and /duplicates.
2. **WinSock REST Server:** Spawns worker threads per socket request, decodes JSON payloads without third-party dependencies, and serializes live engine diagnostics.
3. **Stage 1 Size Bucketing:** Recursively iterates directories. Files are bucketed into an in-memory hash map keyed by exact byte size. Any size bucket with $count == 1$ is immediately discarded.
4. **Stage 2 Worker Pool:** Candidate files sharing sizes are enqueued into a bounded producer-consumer queue. Worker threads hash files concurrently using 64 KB chunks.
5. **Duplicate Clustering:** Worker results are inserted into a hash map keyed by (size, sha256) under a critical section lock. Buckets with ≥ 2 files form final duplicate clusters.

3. Dataset Description (10,000-File Stress Test)

To rigorously evaluate scalability, memory stability, and algorithmic collision resistance under operating-system constraints, a controlled synthetic dataset of **exactly 10,000 files** was generated at C:\thread-scan-stress-10k outside the repository tree. The dataset was constructed to test both positive duplicate detection and negative collision resistance.

Category	File Count	Structure & Size	Algorithmic Role
Unique Files	8,000 files	Sizes 100 to 5,099 bytes across 8 subdirectories	Tests Stage 1 candidate filtering and size bucketing.
Duplicate Pairs	1,000 files	500 pairs of identical files (148 bytes each)	Validates 2-way duplicate detection.
Duplicate Triplets	600 files	200 triplets of identical files (277 bytes each)	Validates multi-way cluster grouping.
Collision Candidates	400 files	200 pairs of identical size (512 bytes), distinct bytes	Validates SHA-256 collision rejection.
Total Dataset	**10,000 files**	**18,323,650 bytes on disk (~17.47 MB)**	**End-to-end stress test**

Reconciliation of Reported Metrics (Recorded Test Results):

• **Why 8,000 files were hashed out of 10,000 discovered:** In the 8,000 unique files, sizes were generated via 100 + ($idx % 5000). Indices 3,000 to 4,999 (2,000 files) produced globally unique sizes; Stage 1 filtered them out without hashing. The remaining 6,000 unique files, 1,000 pair files, 600 triplet files, and 400 collision files all shared sizes with at least one other file, yielding exactly $6000 + 1000 + 600 + 400 = \mathbf{8,000\text{ files processed}}$ ($10,124,650\text{ bytes}$ hashed).

• **Why 752 duplicate groups were formed:** The test planned 700 groups (500 pairs + 200 triplets). The 400 collision files (size 512 bytes) were generated using (65 + ($c % 26)) for 'A' files and (97 + ($c % 26)) for 'B' files ($c \in [0, 199]$). Indices sharing the same remainder modulo 26 had identical bytes, forming 26 duplicate groups of 'A' files and 26 duplicate groups of 'B' files (52 extra groups). Thus, $500 + 200 + 52 = \mathbf{752\text{ duplicate groups}}$ were formed. Crucially, **zero cross-grouping occurred between 'A' and 'B' collision files**.

4. Core Algorithm & Detection Pipeline

The duplicate detection workflow is structured as a two-stage filter that mathematically minimizes computational and I/O overhead:

Stage 1: Recursive Traversal & Size Partitioning: Traverses the target hierarchy via std::filesystem::recursive_directory_iterator with skip_permission_denied. Collects (size, path, timestamp) into an unordered map. This stage performs zero byte-level reads and completes in milliseconds even on 10,000+ files.

Candidate Pruning: Any file whose size key contains only 1 entry is eliminated. In real-world filesystems, this prunes 70–95% of all files immediately.

Stage 2: Parallel Cryptographic Hashing: Candidate files are dispatched to a pre-allocated worker thread pool. Each worker streams file content into a 64 KB RAM buffer and computes an incremental SHA-256 digest.

Collision-Free Group Formation: Completed hashes are inserted into a composite hash map keyed by (size, sha256) under a critical section. Only entries with ≥ 2 matching files are emitted as duplicate clusters.

Storage Reclamation Calculation: For each group of K files each of size S, wasted storage is computed as $S \times (K - 1)$.

5. Actual Source Code Implementation Excerpts

The following excerpts are drawn verbatim from the THREAD//SCAN repository codebase, illustrating core operating-system concurrency and hashing primitives.

File: backend/src/ThreadPool.cpp — *Worker Thread Lifecycle & Condition Variable Coordination*

```
// backend/src/ThreadPool.cpp (Lines 28-60)
while (true) {
    std::function<void(int)> task;
    {
        // Sleep on condition variable until tasks are available or stop requested
        std::unique_lock<std::mutex> lock(m_queueMutex);
        m_cvTaskAvailable.wait(lock, [this]() {
            return m_stop.load() || !m_tasks.empty();
        });

        if (m_stop.load() && m_tasks.empty()) {
            break;
        }

        task = std::move(m_tasks.front());
        m_tasks.pop();
        m_activeWorkers.fetch_add(1);
    } // Queue mutex is released here before executing task!

    try {
        task(workerId); // Work executed outside critical section
    } catch (...) {}

    m_activeWorkers.fetch_sub(1);
    size_t remaining = m_totalTasksRemaining.fetch_sub(1);
    if (remaining == 1) {
        m_cvTasksFinished.notify_all();
    }
}
```

Mechanism Explanation: *Implements a bounded producer-consumer thread pool. Workers sleep on m_cvTaskAvailable without consuming CPU cycles. Crucially, the queue mutex is unlocked before task() execution, ensuring file hashing proceeds in parallel across cores.*

File: backend/src/FileScanner.cpp — *Stage 1 Recursive Filesystem Traversal & Size Bucketing*

```
// backend/src/FileScanner.cpp (Lines 48-78)
fs::path target(rootPath);
if (!fs::exists(target, ec) || !fs::is_directory(target, ec)) {
    return sizeGroups; // Returns empty map gracefully on invalid path
}

fs::directory_options opts = fs::directory_options::skip_permission_denied;
fs::recursive_directory_iterator it(target, opts, ec);
fs::recursive_directory_iterator end;

while (it != end && !ec) {
    try {
        const fs::directory_entry& entry = *it;
        if (entry.is_regular_file(ec) && !entry.is_symlink(ec)) {
            uint64_t size = entry.file_size(ec);
            if (!ec && size >= minSize) {
                FileEntry fe;
                fe.path = entry.path().string();
                fe.sizeBytes = size;
                fe.modifiedAt = formatFileTime(entry.last_write_time(ec));
                sizeGroups[size].push_back(std::move(fe));
                m_filesDiscovered.fetch_add(1, std::memory_order_relaxed);
            }
        }
    } catch (...) {}
    it.increment(ec);
}
```

Mechanism Explanation: *Safely traverses filesystem trees skipping protected files and symlinks. Uses an atomic counter with relaxed memory order for discovered file counts, bucketing files by exact byte size for Stage 1 candidate pruning.*

File: backend/src/FileHasher.cpp — *Chunked 64 KB Cryptographic SHA-256 Streaming*

```
// backend/src/FileHasher.cpp (Lines 11-33)
std::string FileHasher::hashFileSha256(const std::string& filePath) {
    std::ifstream file(filePath, std::ios::binary | std::ios::in);
    if (!file.is_open()) return "";

    SHA256 sha;
    std::vector<char> buffer(BUFFER_SIZE); // 64 KB buffer

    while (file.read(buffer.data(), buffer.size()) || file.gcount() > 0) {
        std::streamsize bytesRead = file.gcount();
        if (bytesRead > 0) {
            sha.update(reinterpret_cast<const uint8_t*>(buffer.data()), static_cast<size_t>(bytesRead));
            m_bytesProcessed.fetch_add(static_cast<uint64_t>(bytesRead));
        }
        if (file.eof()) break;
    }
    return sha.finalize();
}
```

Mechanism Explanation: *Streams file bytes in 64 KB chunks into the SHA-256 state engine. Prevents memory exhaustion when processing multi-megabyte or multi-gigabyte files, while atomically updating total bytes hashed across all concurrent worker threads.*

File: backend/src/DuplicateDetector.cpp — *Task Dispatch, Thread Synchronization & Duplicate Clustering*

```
// backend/src/DuplicateDetector.cpp (Lines 168-194)
for (const auto& file : candidates) {
    if (m_stopRequested.load()) break;

    m_activePool->enqueue([&, file](int workerId) {
        if (m_stopRequested.load()) return;
        m_activePool->recordTaskStart(workerId, file.path);

        std::string hash = hasher.hashFileSha256(file.path);

        if (!hash.empty()) {
            std::lock_guard<std::mutex> lock(hashGroupsMutex);
            GroupKey key{file.sizeBytes, hash};
            hashGroups[key].push_back(file);
        }

        m_filesProcessed.fetch_add(1);
        m_activePool->recordTaskFinish(workerId, file.sizeBytes, elapsed, file.path);
    });
}
m_activePool->waitAll();
```

Mechanism Explanation: *Enqueues lambda task closures into the worker pool. A critical section protected by hashGroupsMutex safely inserts computed hashes into the shared hashGroups map, preventing race conditions during concurrent group formation.*

File: frontend/src/app/duplicates/page.tsx — *Next.js Asynchronous Duplicate Cluster Hydration & Rendering*

```
// frontend/src/app/duplicates/page.tsx (Excerpt)
const { duplicateGroups, fetchLatestDuplicates, backendOnline } = useScanner();

useEffect(() => {
    if (backendOnline) {
        fetchLatestDuplicates(); // Hydrates duplicate clusters from GET /api/v1/duplicates
    }
}, [backendOnline, fetchLatestDuplicates]);

return (
    <div className="space-y-4">
        {duplicateGroups.map((group) => (
            <div key={group.id} className="p-4 rounded border border-zinc-800 bg-zinc-900/60">
                <div className="flex justify-between items-center text-xs">
                    <span className="font-mono text-emerald-400">{group.id} ({group.fileCount} files)</span>
                    <span className="text-zinc-400">Wasted: {formatBytes(group.sizeBytes * (group.fileCount -
 1))}</span>
                </div>
                <div className="mt-2 text-xs font-mono text-zinc-500 break-all">{group.hash}</div>
            </div>
        ))}
    </div>
);
```

Mechanism Explanation: *React hook polls GET /api/v1/duplicates from C++ backend, rendering verified duplicate clusters, cryptographic hashes, file lists, and wasted byte calculations in an industrial dark-mode workstation dashboard.*

6. Empirical Verification & Test Results

All test cases were executed directly against the compiled C++17 engine on the physical Windows NT test environment. No results have been manufactured or assumed.

Metric Parameter	Recorded Test Result	Algorithmic Significance
Files Discovered	**10,000 files**	Recursive traversal enumerated 100% of directory tree.
Files Processed (Stage 2)	**8,000 files**	2,000 unique-size files successfully pruned in Stage 1.
Bytes Cryptographically Hashed	**10,124,650 bytes (~9.65 MB)**	Matches mathematical sum of candidate file sizes.
Duplicate Groups Emitted	**752 groups**	500 pairs + 200 triplets + 52 modulo-26 collision groups.
Total Redundant Files	**2,000 files**	1,000 pair files + 600 triplet files + 400 collision files.
Wasted Storage Identified	**367,946 bytes (~359.3 KB)**	Accurately sums size * (count - 1) across all groups.
Total Execution Duration	**2,916 ms (~2.92 seconds)**	Sub-3-second throughput across 8 worker threads.

Comprehensive Test Register (Evidence Mapped):

Test ID	Test Category & Objective	Outcome	Evidence Artifact
TC-BLD-01	C++17 Static Backend Compilation (GCC 14.2.0)	**PASS**	thread_scan_server.exe (3.34 MB)
TC-BLD-02	Next.js Production Lint & Turbopack Build	**PASS**	10 static prerendered routes
TC-DIR-01	Normal Recursive Directory Traversal	**PASS**	api_scan_metrics_normal.json
TC-DUP-01	Exact Duplicate Pair Detection	**PASS**	api_duplicates_multi4.json
TC-COL-01	Collision Resistance (Same Size, Diff Bytes)	**PASS**	api_duplicates_multi4.json
TC-MOD-01/02	Single-Thread vs. 4-Thread Pool Parity	**PASS**	1.79x speedup, 0 hash divergence
TC-BEN-01	Multithreaded Benchmark (1, 2, 4, 8 Threads)	**PASS**	api_benchmark_trial2.json
TC-ERR-01	Non-Existent Target Path Handling	**PASS**	api_scan_metrics_invalid.json
TC-LGC-01	10,000-File High-Density Stress Test	**PASS**	screenshot_stress_10k_overview.png
TC-CON-01	Concurrent File Mutation During Active Scan	*NOT RUN*	Requires fault injection harness

7. Setup and Execution Guide

The following build and launch commands are derived directly from the verified project toolchain and scripts on Windows PowerShell:

File: Windows PowerShell — *Compilation, Build & Production Launch Pipeline*

```
# 1. Compile C++17 Backend Daemon (MinGW-W64 GCC):
& "C:\Program Files\CodeBlocks\MinGW\bin\g++.exe" -std=c++17 -O2 -static -static-libgcc -static-libstdc++ `
  -Ibackend/src backend/src/main.cpp backend/src/Sha256.cpp backend/src/ThreadPool.cpp `
  backend/src/FileScanner.cpp backend/src/FileHasher.cpp backend/src/DuplicateDetector.cpp `
  backend/src/Benchmark.cpp backend/src/ApiServer.cpp -o backend/thread_scan_server.exe -lws2_32

# 2. Run Backend Daemon (Default Port 8080):
.\backend\thread_scan_server.exe

# 3. Install Frontend Dependencies & Start Dashboard (Port 3000):
npm --prefix frontend install
npm --prefix frontend run build
npm --prefix frontend run start
```

Mechanism Explanation: *Static compilation with -static-libgcc -static-libstdc++ -lws2_32 creates a portable zero-dependency Windows binary. The frontend connects to http://localhost:8080 by default (configurable via NEXT_PUBLIC_API_URL).*

8. Operating System Concepts & Conclusion

Core Operating System Principles Demonstrated:

- **Bounded Thread Pools vs. Thread-per-Task:** Eliminates kernel thread thrashing and unbounded context-switching overhead.
- **Condition Variables (POSIX / Win32 CV):** Ensures idle workers remain blocked without consuming scheduler quantum.
- **Critical Section Granularity:** I/O and SHA-256 computation are executed outside lock guards; mutexes protect only task dequeue and map insertion.
- **Filesystem Buffer Cache Dynamics:** Benchmark analysis demonstrates Amdahl's Law scalability ceilings ($p \approx 85.4\%, S_\infty \approx 6.85\times$) and OS page cache priming effects.

Conclusion: THREAD//SCAN successfully demonstrates an industrial-grade, concurrent filesystem analyzer capable of processing 10,000 files in under 3 seconds with zero race conditions, verified collision resistance, and complete operating-system telemetry.